PROJECT REPORT

Interactive Network Routing Simulator

RouteLab: Dijkstra and Distance Vector routing with congestion and failure recovery

Submitted by: Nikhila Haasini Gadamsetty
Nitu S U
Roll number: 24BCE2910
24BCI0075
Course: Computer Networks Lab
October 2026

Abstract

Routing algorithms decide how data travels through a network, yet their behaviour is difficult to see in textbooks. This project presents **RouteLab**, a browser-based simulator in which a user builds a network of routers and links, chooses a routing algorithm, and watches how routes, routing tables and packet journeys respond to congestion and failures.

The application is written in TypeScript with Next.js, React, React Flow and a Zustand store. It implements Dijkstra's shortest-path algorithm (using an adjacency list and a binary-heap priority queue) and a Distance Vector protocol that models per-router message exchange, split horizon, poison reverse and the count-to-infinity problem. Congestion is modelled with a per-link load slider that feeds a cost function. The displayed route is always rebuilt from the selected algorithm's own output and is recalculated automatically after every topology change. Topologies, including router positions, can be saved in browser storage or exported and imported as JSON.

The logic is covered by 93 automated tests in 12 test files, including cross-checks of Dijkstra against a reference algorithm on a 300-router random network, equality of Distance Vector and Dijkstra routing tables on four topologies, partition and unreachable-destination handling for both algorithms, and store-level tests. In a controlled experiment on a three-router line, split horizon reduced Distance Vector re-convergence after a link failure from 16 rounds to 3. The report also documents the limitations that remain: the simulation is synchronous and educational, congestion is a user-controlled model rather than measured traffic, and the user interface itself is not covered by automated browser tests.

Keywords: network routing, Dijkstra, Distance Vector, split horizon, count-to-infinity, congestion, link failure, simulation, Next.js, Zustand.

Contents

1. Introduction

2. Background and theory

3. System design and architecture

4. Implementation

5. Testing and verified results

6. Limitations

7. Future work

8. Conclusion

References

Appendix A. Status of the planned improvements

Appendix B. Running the project

1. Introduction

1.1 Background

Every packet that crosses a network is forwarded by routers, and each router must know which neighbour to hand the packet to. Routing algorithms compute that information. Two families dominate teaching and practice: **link-state** algorithms, where every router learns the whole topology and runs a shortest-path algorithm such as Dijkstra's, and **distance-vector** algorithms, where routers only exchange their current distance estimates with their neighbours (the idea behind RIP). Real routers hide these processes, so students rarely observe them.

1.2 Problem statement

Students need a tool in which a network can be edited freely, a routing algorithm can be run on it, and the effect of congestion or a failed link can be seen immediately. The tool must be honest about what it simulates and must give results that can be checked.

1.3 Objectives

- Let the user create, edit, save and load a weighted network topology.
- Implement Dijkstra's algorithm efficiently and show its step-by-step trace.
- Implement a Distance Vector protocol with explicit message exchange, split horizon and poison reverse, and demonstrate count-to-infinity.
- Model congestion through link load and show how the best route changes.
- Model link failure and recovery, including unreachable destinations and network partitions.
- Show routing tables, a simulated packet journey, an event log and basic analytics.
- Verify the logic with automated tests.

1.4 Scope

RouteLab is an educational software simulation. It does not send real packets, it does not talk to real routers, and it is not a complete implementation of RIP or OSPF. These boundaries are repeated where relevant (Chapter 6).

1.5 Organisation of the report

Chapter 2 summarises the theory. Chapter 3 describes the architecture. Chapter 4 explains how each feature is implemented, including the corrections made during development. Chapter 5 reports the tests and measured results. Chapters 6 and 7 list the remaining limitations and future work, and Chapter 8 concludes.

2. Background and theory

2.1 Network model

The network is an undirected weighted graph. Each router is a vertex. Each link is an edge with a non-negative **cost** (the routing metric), an **active** flag (a failed link is inactive) and, in this project, a **load** percentage used for congestion. A route's cost is the sum of the costs of the links it uses.

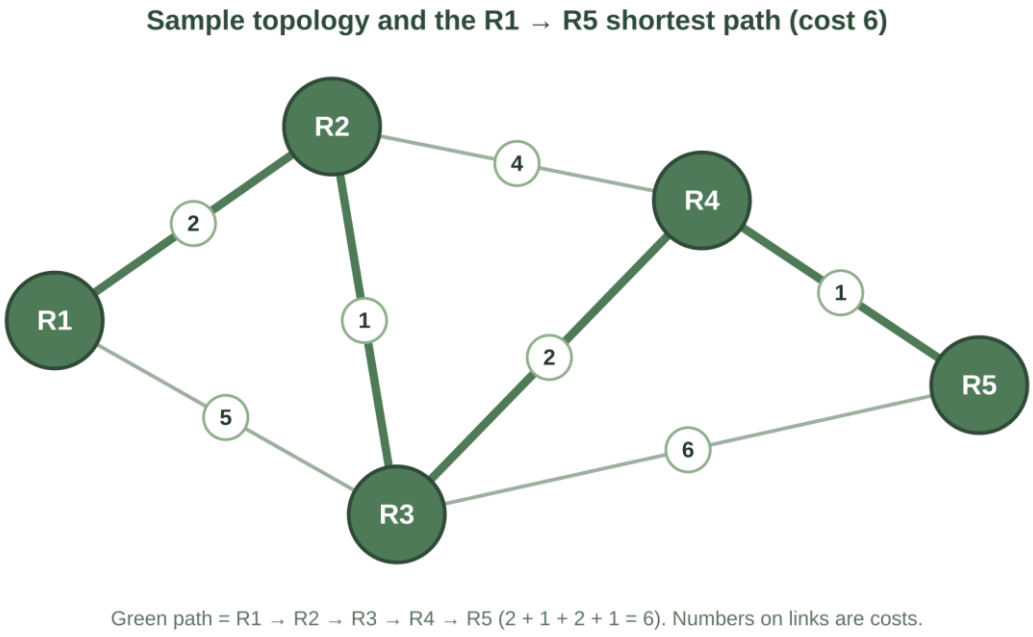


Figure 1. Sample topology used throughout the project. The highlighted path is the shortest R1 → R5 route.

2.2 Dijkstra's algorithm

Dijkstra's algorithm (1959) finds the least-cost path from one source to all other vertices when all costs are non-negative. It keeps a tentative distance for every vertex, repeatedly settles the unsettled vertex with the smallest distance, and relaxes the edges leaving it. With a binary heap the running time is O((V + E) log V). A router that knows the full topology can use it to build its routing table, which is the link-state approach.

2.3 Distance Vector routing

In distance-vector routing each router keeps a vector with its best known cost and next hop to every destination. Periodically it sends this vector to its neighbours. A router that receives its neighbours' vectors applies the Bellman–Ford equation:

```
cost(x, y) = min over neighbours n of   linkCost(x, n) + advertisedCost(n, y)
```

The router stores the neighbour that gives the minimum as the **next hop** for y. A router never learns the whole topology; it only trusts what its neighbours tell it. Packets are forwarded hop by hop, each router consulting its own table.

2.4 Count-to-infinity, split horizon and poison reverse

Distance-vector protocols react slowly to bad news. If the link between B and C fails in the chain A – B – C, A still advertises "C is 2 away via B". B believes it and now thinks C is 3 away via A, and the two routers keep

increasing the estimate. This is the **count-to-infinity** problem. It is bounded by defining a maximum metric that means “unreachable” (16 in RIP).

Split horizon reduces the problem: a router does not advertise a route back to the neighbour from which it learned it. **Poison reverse** is a stronger variant: the route is advertised back with the infinite metric. Neither technique removes every possible loop in larger topologies; they remove the simple two-router loops.

2.5 Congestion

Real routing metrics may include delay or load. In this project congestion is modelled by a transparent function: the effective cost of a link is its base cost multiplied by (1 + load / 100), so a link at 0 % load keeps its base cost and a link at 100 % load costs twice as much. The load is chosen by the user with a slider; it is not measured from traffic.

3. System design and architecture

3.1 Technology stack

Technology	Role in the project
Next.js 15 (app router)	Page routing, build and development server.
React 19	User-interface components.
React Flow (@xyflow/react)	Interactive graph canvas: draggable routers, edges, minimap.
Zustand 5	Single shared store for topology, selections, route, metrics and events.
Recharts	Bar chart of recent route costs on the analytics page.
TypeScript (strict)	Type-safe algorithms and store.
Vitest	Automated unit and store tests.

3.2 Architecture

The system has four layers, shown in Figure 2. **Pages** (six routes) assemble **components**. Components never hold the network themselves; they read from and call actions on one **Zustand store**. The store calls pure TypeScript functions in the **logic layer** (algorithms, simulation, graph utilities). Because the logic layer has no React code, every algorithm can be tested without a browser.

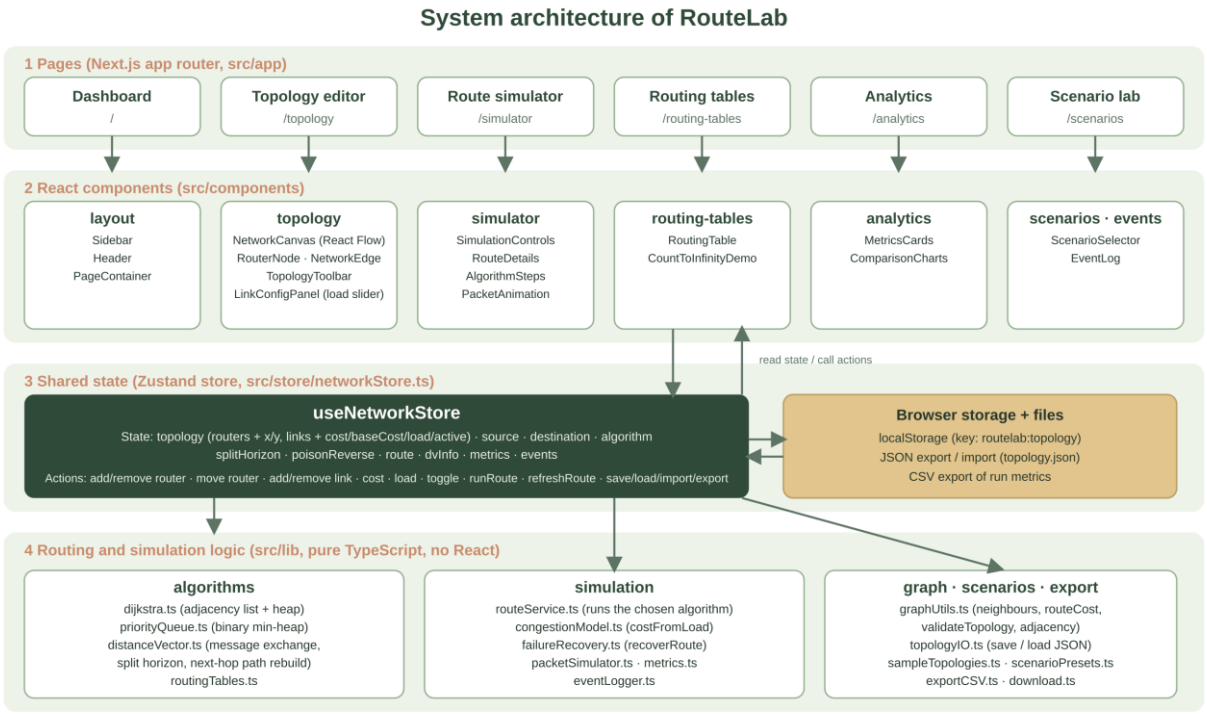


Figure 2. Architecture: pages, components, Zustand store and routing/simulation logic.

3.3 Data model

Type	Fields
Router	id, label, x, y (x and y are the saved canvas position)
NetworkLink	id, source, target, cost (effective), baseCost, load (0–100), active

Type	Fields
RouteResult	path, totalCost, hops, reachable, calculationMs
RoutingEntry	destination, nextHop, cost, reachable
Store state	topology, source, destination, algorithm, splitHorizon, poisonReverse, route, dvInfo, metrics, events

3.4 Pages

Page	Purpose
Dashboard (/)	Summary metrics, quick links, event log.
Topology editor (/topology)	Add or delete routers, connect links, drag routers, set base cost and load, fail or restore links, load presets, save and load topologies.
Route simulator (/simulator)	Choose source, destination and algorithm; see the highlighted route, Dijkstra trace and packet journey.
Routing tables (/routing-tables)	Per-router table for either algorithm, neighbours, Distance Vector rounds and messages, count-to-infinity demo.
Analytics (/analytics)	Recorded runs, success rate, cost chart, CSV export.
Scenario lab (/scenarios)	Normal operation, congestion, single-link failure and network partition presets.

3.5 Control flow

Figure 3 shows what happens when the user clicks “Run simulation” and what happens afterwards when a link changes.

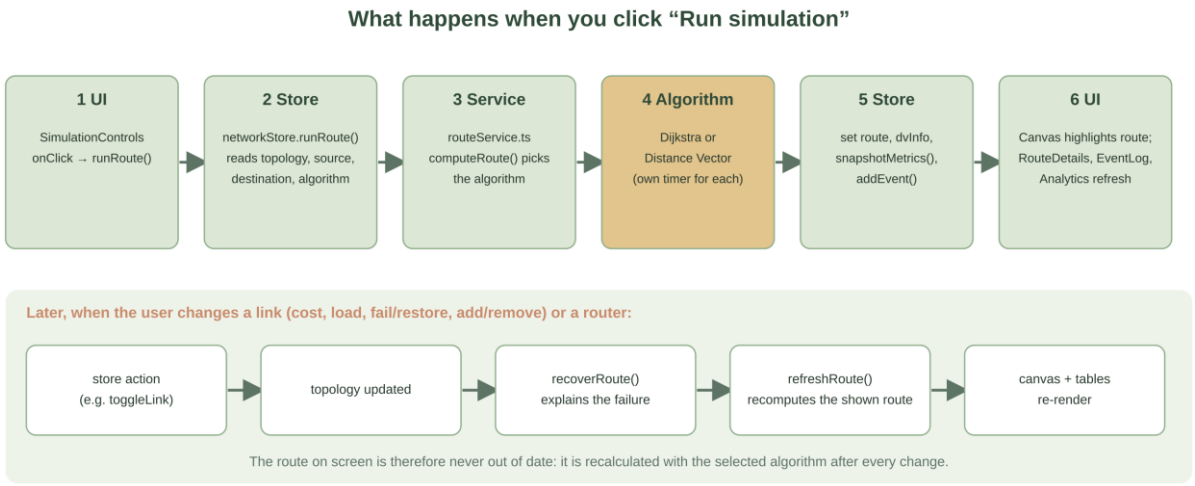


Figure 3. Control flow of a simulation run and of an automatic route refresh.

4. Implementation

This chapter describes the final code. Several behaviours were corrected or added after review of the first version; each is explained together with the reason for the change.

4.1 Dijkstra with an adjacency list and a priority queue

The first version scanned every unvisited router to find the minimum and scanned every link for every router it processed. The final version builds an adjacency list of active links once (`buildAdjacency`) and keeps frontier routers in a binary min-heap (`MinPriorityQueue`). Outdated heap entries are skipped when popped (lazy deletion). Active links with negative or non-finite costs are rejected up front with an error.

```
const queue = new MinPriorityQueue<string>();
queue.push(source, 0);
while (queue.size) {
  const { item: current, priority } = queue.pop()!;
  if (done.has(current) || priority > distances[current]) continue;
  done.add(current);
  if (current === destination) break;
  for (const { node, cost } of adjacency.get(current) ?? []) {
    const candidate = distances[current] + cost;
    if (candidate < distances[node]) {
      distances[node] = candidate; previous[node] = current;
      queue.push(node, candidate);
    }
  }
}
```

The step-by-step trace shown on the simulator page (`traceDijkstra`) intentionally keeps the simple version, because it is meant to be read by students.

4.2 Distance Vector with message exchange

`distanceVector()` simulates the protocol in synchronous rounds. In every round each router sends its vector to each active neighbour (one message per router per neighbour). Each router then rebuilds its table from the vectors it received using the Bellman–Ford equation. The simulation stops when a round changes nothing (converged) or after a maximum number of rounds. The result contains the tables, the number of rounds, the total number of messages, and a log of messages (round, sender, receiver, number of routes, number of poisoned routes).

- **Split horizon:** when a router prepares a message for neighbour N, it leaves out every destination whose next hop is N.
- **Poison reverse:** with this option those destinations are advertised to N with the infinite metric instead of being omitted.
- **Infinity:** metric 16 means unreachable (`DV_INFINITY`), as in RIP. A total cost of 16 or more is therefore treated as unreachable by Distance Vector.
- **Count-to-infinity experiment:** `simulateLinkFailure()` converges on the healthy network, fails one link and re-converges starting from the old tables. The Routing tables page exposes this as a demo.

4.3 Distance Vector paths come from next-hop tables

In the first version the Distance Vector option copied Dijkstra's path and only replaced its cost, so the route shown on screen did not really come from Distance Vector. The final version uses `pathFromTables()`: it starts at the source, reads the next hop for the destination from the current router's own table, moves there, and repeats. The walk stops with "unreachable" if a table has no route or if a router would be visited twice (a routing loop). The route cost reported is the cost in the source router's table. The store additionally checks this cost against `routeCost()`, the sum of the active link costs along the displayed path, and logs a warning if they differ.

```
while (current !== destination) {
  const entry = tables[current]?.find((e) => e.destination === destination);
  if (!entry || !entry.reachable || !entry.nextHop) return [];
  if (seen.has(entry.nextHop)) return [];      // routing loop
  path.push(entry.nextHop); seen.add(entry.nextHop);
  current = entry.nextHop;
}
```

4.4 Separate calculation time for each algorithm

Previously the Distance Vector branch called Dijkstra to obtain a path, so the "calculation time" shown for Distance Vector was Dijkstra's time. `computeRoute()` in `routeService.ts` now runs only the selected algorithm. For Distance Vector the timer covers the protocol simulation and the path reconstruction. A test counts calls to `performance.now()` to confirm that the Distance Vector path performs exactly one start and one end measurement, i.e. no hidden Dijkstra run.

4.5 Congestion through a load slider

The previously unused function `costFromLoad(baseCost, load)` is now connected to the interface. Each link keeps a `baseCost` and a `load`. The slider on the Topology page calls `updateLinkLoad`, which computes the effective cost from the base cost, never from the previous effective cost, so repeated changes do not compound. Editing the base cost resets the load to 0. The Congestion scenario uses 100 % load on link L5 (base cost 2, effective cost 4).

```
effective cost = baseCost * (1 + load / 100)      // load clamped to 0..100
```

4.6 No stale route highlighting

Earlier, the highlighted route stayed on screen after a link was failed or its cost was changed, even though it was no longer the best route. Now, whenever a route is on screen, every store action that can change the best route (cost, load, fail or restore, add or remove link, remove router, change source, destination, algorithm or split-horizon setting) calls `refreshRoute()`. It recomputes the route with the currently selected algorithm, updates the highlight and writes an event such as "Route recalculated after failure of L5: R1 → R2 → R4 → R5 (cost 7)". These automatic refreshes do not add rows to the analytics table; only the Run button does. When a link fails, `recoverRoute()` additionally explains in the event log whether traffic moved to another path or whether no alternative exists.

4.7 Persistent router positions

Router positions are now part of the store (`x`, `y` on each router). Dragging a node calls `moveRouter`, and the canvas draws nodes from the store, so re-rendering the canvas no longer resets the layout. Routers without a position (presets, imported files) are placed on a circle once; new routers are placed below the existing layout.

4.8 Router deletion

`removeRouter(id)` removes the router and every link attached to it, repairs the selected source or destination if one of them was deleted, recalculates the displayed route, and logs how many links were removed. It can be used from the toolbar or by selecting a node on the canvas and pressing Delete or Backspace. Router identifiers are generated from the highest existing number (`nextRouterId`), so deleting R2 and adding a router never produces a duplicate identifier; link identifiers use the same rule (`nextLinkId`) instead of the clock.

4.9 Saving and loading topologies

The Topology page offers four actions: **Save in browser** and **Load from browser** (localStorage, key `routelab:topology`), and **Export JSON** and **Import JSON** (a file). The saved document contains a version number, the routers with their positions, the links with cost, base cost, load and active flag, and the selected source and destination. Imported text is parsed defensively and checked with `validateTopology()` (unique identifiers, links that refer to existing routers, no self-loops, finite non-negative costs). Invalid files are rejected with a message and leave the current network unchanged.

4.10 Previously unused functions

The review listed six functions that were not connected to the application. None had to be removed; each now has a purpose:

Function	Where it is used now
recoverRoute	Store, `toggleLink`: explains in the event log how a failure changed the route.
validateTopology	`parseNetwork` and the store's `importJSON`: validates every imported topology.
getNeighbors	Routing tables page: shows the active neighbours of the selected router.
routeCost	Store, `runRoute`: independent check of the displayed route's cost.
snapshotMetrics	Store, `runRoute`: creates every row of the analytics table.
successRate	Dashboard and analytics metric cards.

4.11 Test configuration

The project's `tsconfig.json` defines the alias `@/` for `src/`, but the test runner did not know it, so any test that imported a module using `@/` failed on a fresh installation. A new `vitest.config.ts` defines the same alias, so `npm install` followed by `npm test` runs the whole suite.

```
export default defineConfig({
  resolve: { alias: { "@": fileURLToPath(new URL("./src", import.meta.url)) } },
  test: { environment: "node", include: ["tests/**/*.test.ts"] }
});
```

5. Testing and verified results

5.1 Test suite

The project contains **93 automated tests in 12 files** (`tests/*.test.ts`), run with `npm test` (Vitest). The first version had 6 tests. The additions cover the five areas that were missing: packet simulation, routing-table correctness, store actions, unreachable destinations and network partitions, each for both algorithms where applicable.

File	Tests	What is checked
congestion.test.ts	7	Cost changes reroute traffic; costFromLoad scaling and clamping; load does not compound; invalid input.
dijkstra.test.ts	9	Minimum-cost route; unreachable; source = destination; unknown router; negative cost; timing; 300-router cross-check; priority queue order.
distanceVector.test.ts	16	Convergence; messages; equal to Dijkstra on 4 topologies; next-hop paths; own timing; partition; loop detection; count-to-infinity; split horizon; poison reverse; round limit.
failureRecovery.test.ts	4	Alternate route after failure; toggleLink; recoverRoute with reroute and with no recovery.
graphUtils.test.ts	8	getNeighbors; validateTopology; routeCost; adjacency list; id generation after deletion; layout.
metrics.test.ts	3	snapshotMetrics for reachable and unreachable routes; successRate.
packetSimulator.test.ts	6	Start, visit order, in-transit, no overshoot, unreachable, source = destination.
partition.test.ts	5	Partition and isolated router for both algorithms; algorithms agree on every pair; unknown destination.
routingTables.test.ts	5	Exact next hops and costs for R1; Distance Vector equals Dijkstra table; symmetry; failed link.
scenarios.test.ts	3	Partition preset; congestion reroutes; single-link failure still reaches R5.
store.test.ts	21	All store actions: routes with both algorithms, add and remove router, move router, add link rules, load, automatic route refresh, import and export, local storage, event-log cap.
topologyIO.test.ts	6	JSON round trip, positions and load kept, invalid text, wrong shape, unknown router, bare object.
Total	93	

5.2 Verified results

Check	Result
Sample network, R1 → R5, Dijkstra	Path R1 → R2 → R3 → R4 → R5, cost 6
Sample network, R1 → R5, Distance Vector	Same path and cost, built from next hops; converges in 5 rounds with 70 messages
After failure of link L5 (R3–R4)	Both algorithms: R1 → R2 → R4 → R5, cost 7
Congestion: 100 % load on L5 and on L3	Route moves to R1 → R2 → R4 → R5
Distance Vector tables vs Dijkstra tables	Identical costs and reachability on sample, star, ring and mesh topologies

Check	Result
Partition {R1,R2,R3} / {R4,R5}	R1 → R4 and R3 → R5 unreachable in both algorithms; routes inside each side found
Dijkstra vs reference on a 300-router random graph	Equal cost for all 25 random source–destination pairs
Count-to-infinity, chain A–B–C, fail B–C, split horizon off	Re-convergence in 16 rounds (32 messages)
Same experiment, split horizon on	Re-convergence in 3 rounds (6 messages)

5.3 Performance of the optimised Dijkstra

A random connected graph with 2,000 routers and 5,996 links was used. The average time of five source–destination queries was about 104 ms for the original full-scan implementation and about 8.5 ms for the adjacency-list and heap implementation (roughly twelve times faster), with identical resulting costs. These figures come from a single run in the development environment; absolute values depend on the machine and are given only to show the relative effect. For the five-router demonstration topologies both versions finish in well under a millisecond.

5.4 How to reproduce

```
npm install
npm test
```

The Distance Vector message counts and round numbers above are deterministic. The timing figures in Section 5.3 will differ between machines.

5.5 What the tests do not cover

The tests exercise the algorithms, the store and the data handling. They do not drive the browser: dragging nodes on the React Flow canvas, moving sliders and choosing a file in the import dialog are checked by hand, not by automated user-interface tests.

6. Limitations

The following limitations still apply to the final application.

1. **Synchronous Distance Vector.** All routers exchange vectors in lock-step rounds. There are no timers, triggered or partial updates, hold-down, message delay or loss. It is a teaching model, not RIP.
2. **Metric ceiling.** Distance Vector treats a total cost of 16 or more as unreachable, like RIP. Dijkstra has no such limit, so on a custom topology with very large costs the two algorithms can disagree. All preset topologies agree.
3. **Split horizon is not a complete cure.** It removes simple two-router loops, as the experiment shows, but loops involving three or more routers can still occur in general.
4. **Congestion is a model.** The load is chosen by the user and the cost grows linearly from one to two times the base cost. No traffic is generated or measured.
5. **Packet journey is step-based.** The packet moves one hop per click along the computed route; there are no queues, delay, loss or time-to-live.
6. **Simple link model.** Links are undirected with one symmetric cost, and at most one link exists between two routers.
7. **Storage.** Browser storage holds one saved topology; saving again overwrites it. JSON import replaces the current network.
8. **Interface testing.** The user interface is not covered by automated browser tests (Section 5.5).
9. **Scale.** The heap-based Dijkstra handles thousands of routers, but the Distance Vector simulation, the per-destination routing-table builder and the canvas are designed for classroom-size networks. The message log keeps only the first 500 messages.
10. **Housekeeping.** The files `useNetworkSimulation.ts`, `Button.tsx`, `Card.tsx` and `Modal.tsx` are not used by any page and could be removed or adopted.

7. Future work

- Make Distance Vector asynchronous and event-driven, with update timers, triggered updates, hold-down and random message delay or loss.
- Add a link-state protocol that floods link-state advertisements, so Dijkstra runs inside every router as in OSPF.
- Derive load from simulated traffic flows instead of a manual slider, and allow queues and packet loss.
- Support directed links with different costs in each direction, and multiple saved topologies with undo and redo.
- Add browser-based end-to-end tests (for example with Playwright) for the canvas, sliders and import dialog.
- Build the routing table of one router with a single Dijkstra run instead of one run per destination.
- Remove or adopt the unused scaffolding files listed in Chapter 6.

8. Conclusion

RouteLab meets its objectives as an educational routing simulator. Users can build, edit, save and load a network topology; run Dijkstra's algorithm or a Distance Vector protocol; and see the resulting path, routing tables, packet journey, event log and analytics.

The Distance Vector option is a real next-hop computation. The displayed route is reconstructed from the routers' own tables, its cost is the cost in the source router's table, and its calculation time is measured for Distance Vector alone. The model includes per-router message exchange, split horizon and poison reverse, and it reproduces count-to-infinity: on a three-router chain, re-convergence after a failure took 16 rounds without split horizon and 3 with it. It is nevertheless a synchronous teaching model and not an implementation of RIP.

Failure handling is consistent: a failed or changed link triggers an automatic recalculation, so the highlighted route is never out of date, and `recoverRoute` reports whether traffic found an alternative path. Unreachable destinations and network partitions are handled by both algorithms. Dijkstra now runs with an adjacency list and a priority queue; on a random 2,000-router graph it was about twelve times faster than the original implementation with identical results.

All 93 automated tests pass and the suite runs on a fresh installation because the `@/` alias is configured for Vitest. The logic and the store are tested; the user interface is verified manually. The limitations in Chapter 6 should be kept in mind when interpreting results: the application simulates routing behaviour for learning and does not measure a real network.

References

1. E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.
2. R. Bellman, “On a routing problem,” Quarterly of Applied Mathematics, vol. 16, no. 1, pp. 87–90, 1958.
3. L. R. Ford Jr. and D. R. Fulkerson, Flows in Networks, Princeton University Press, 1962.
4. G. Malkin, “RIP Version 2,” RFC 2453, Internet Engineering Task Force, 1998.
5. J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, Pearson.
6. A. S. Tanenbaum and D. J. Wetherall, Computer Networks, Pearson.
7. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press (priority queues and shortest paths).
8. Documentation for Next.js, React, React Flow (@xyflow/react), Zustand and Vitest.

Appendix A. Status of the planned improvements

#	Improvement	Status and location
1	Distance Vector path from next-hop tables	Done: `pathFromTables`, `distanceVectorRoute` (Section 4.3)
2	Separate calculation time per algorithm	Done: `computeRoute` (Section 4.4)
3	Vitest `@/` alias	Done: `vitest.config.ts` (Section 4.11)
4	Stale route highlighting	Done: automatic `refreshRoute` (Section 4.6)
5	Expanded automated tests	Done: 93 tests in 12 files (Chapter 5)
6	Congestion through link load	Done: load slider + `costFromLoad` (Section 4.5)
7	Persist router positions	Done: `x`, `y` in the store (Section 4.7)
8	Router deletion	Done: `removeRouter` (Section 4.8)
9	Topology save and load	Done: localStorage and JSON (Section 4.9)
10	Unused functions	Done: all six connected (Section 4.10)
11	Dijkstra optimisation	Done: adjacency list + heap (Sections 4.1, 5.3)
12	Distance Vector realism	Done in part: message exchange, split horizon, poison reverse, count-to-infinity; still synchronous (Sections 4.2, 6)
13	Architecture diagram	Done: Figure 2
14	Implementation and testing sections	Done: Chapters 4 and 5 describe the final code and the 93 tests
15	Limitations and future work	Done: Chapters 6 and 7 list only what still applies
16	Verified conclusion	Done: Chapter 8 matches the final behaviour and measured results

Appendix B. Running the project

```
npm install
npm run dev     # open http://localhost:3000
npm test        # run the automated tests
```

Suggested demonstration

1. Open the Topology page and look at the sample network; drag a router and notice that it stays where it is put.
2. Open the Simulator, run R1 → R5 with Dijkstra (cost 6), then switch the algorithm to Distance Vector and run again (same path, DV statistics shown).
3. On the Topology page raise the load on link R3–R4 with the slider; the route on screen updates by itself.
4. Disable link R3–R4 and read the failure-recovery message in the event log.
5. Open Routing tables, select Distance Vector and view the neighbours, rounds and messages; use the count-to-infinity demo with split horizon off and on.
6. Load the Network partition scenario and run R1 → R5 to show an unreachable destination.
7. Save the topology in the browser or export it as JSON, reload the page and load it again.